



Rapport de projet long : Technologie Objet

Noé Rigoletti - Raphaël Landry - Desesquelles Lilian - Amedro Alexandre
- Puech Landry - Magron Mathis - Dissoubray Alice - Rouzaud Camille

Année universitaire 2024/2025

Table des matières

1	Introduction	2
2	Principales fonctionnalités (User Stories et État de réalisation)	2
3	Découpage en sous-systèmes	3
4	Diagramme de classe	4
5	Dynamique du système	4
5.1	Déplacement du joueur sur la carte	4
5.2	Ouverture d'une porte	5
5.3	Utilisation d'un objet de l'inventaire	5
5.4	Combat contre un sbire	5
5.5	Mort du joueur	5
6	Choix de conception, problèmes rencontrés et solutions	5
6.1	Choix de conception	5
6.2	Problèmes rencontrés et solutions apportés	6
7	Organisation de l'équipe et méthodes agiles	8
7.1	Organisation	8
7.1.1	Contribution de Mathis	8
7.1.2	Contribution de Raphaël	8
7.1.3	Contribution de Noé	9
7.1.4	Contribution de Lilian	9
7.1.5	Contribution d'Alexandre	9
7.1.6	Contribution de Julian	10
7.2	Méthodes agiles	10
7.2.1	Réunions Hebdomadaires	10
7.2.2	Planification Itérative (Sprints)	10
7.2.3	Rétrospectives de Sprint	10
7.2.4	Gestion du Backlog	10
7.2.5	Collaboration et Communication	10
7.3	Défis et Apprentissages	11
7.3.1	Défis Rencontrés	11
7.3.2	Apprentissages	11
8	Conclusion	11

1 Introduction

L'objectif du projet est de développer en Java un jeu vidéo de type Rogue-Like en 2D vue du dessus, dans un univers mythologique. Le joueur incarne Hercule et doit progresser à travers des salles générées aléatoirement, vaincre des monstres et des boss représentant les 12 travaux, en apprenant de ses échecs pour avancer. Le gameplay repose en principe sur la répétition et l'apprentissage : le joueur est amené à échouer, à reconnaître les schémas d'attaque ennemis et à améliorer son personnage. Le jeu intègre des mécaniques d'amélioration temporaire (monnaie en jeu) et permanente (expérience), une gestion de l'inventaire, de la santé, du bouclier, et de la mana, ainsi qu'un système de combat à l'arme blanche ou à distance.

Pour mener à bien ce projet, nous avons utilisé LibGDX qui est un framework Java spécialisé dans le développement de jeux vidéo. Cette bibliothèque nous a permis de gérer efficacement différents aspects techniques du jeu tels que l'affichage graphique en 2D, la détection des collisions, les animations, la gestion des entrées clavier/souris et que l'organisation des différentes scènes du jeu (menu, niveaux, inventaire, etc.). Cette librairie nous a donc permis de développer le jeu, en facilitant l'intégration des différentes fonctionnalités.

2 Principales fonctionnalités (User Stories et État de réalisation)

Lors du développement du jeu, nous avons structuré les besoins sous forme de parties. Chacune a été planifiée sur plusieurs itérations agiles.

User Story / Epic	État	Itération
En tant que joueur, je peux me déplacer librement dans une map.	Réalisée	Itération 1
En tant que joueur, je peux voir la carte générée procéduralement (salles et couloirs).	Réalisée	Itération 1
En tant que joueur, je peux ouvrir une porte en m'en approchant.	Réalisée	Itération 2
En tant que joueur, je peux ramasser des objets dans un coffre.	Réalisée	Itération 3
En tant que joueur, je peux ouvrir un coffre uniquement s'il n'a pas déjà été ouvert.	Réalisée	Itération 3
En tant que joueur, je peux combattre des ennemis présents dans certaines salles et des boss.	Réalisée	Itération 3
En tant que joueur, je peux accéder à mon inventaire pour voir les objets collectés.	Réalisée	Itération 3
En tant que joueur, je peux voir une animation lorsque j'attaque ou suis attaqué.	Réalisée	Itération 3
En tant que joueur, je peux revenir au menu principal depuis le jeu.	Réalisée	Itération 2
En tant que joueur, je peux démarrer une nouvelle partie depuis le menu.	Réalisée	Itération 2
En tant que joueur, je compléter les 12 travaux d'Hercule et fini le jeu	à faire	Itération 4

TABLE 1 – User stories principales, leur état de réalisation et l'itération associée

3 Découpage en sous-systèmes

Afin d'assurer une meilleure organisation du code, une maintenabilité accrue et une séparation claire des responsabilités, l'application a été découpée en plusieurs paquetages (ou sous-systèmes). Chaque paquetage regroupe les classes ayant des rôles similaires dans le fonctionnement du jeu.

- **inventaire** : Gère les objets ramassés par le joueur. Ce paquetage contient les classes liées aux items, aux équipements et à la gestion de l'inventaire. Il inclut également les classes comme `Item`, `Coffre`, et `DatabaseItem`.
- **map** : Représente la carte du jeu, les salles (chambres), les couloirs et la génération procédurale. Il regroupe des classes comme `Map`, `Chambre`, et leurs outils de construction de niveaux.
- **menu** : Contient les interfaces utilisateur (UI) pour le menu principal, les écrans de pause, de victoire ou de défaite. Ce sous-système gère également la navigation entre les différentes scènes du jeu.
- **animation** : Ce paquetage encapsule les classes responsables des animations du personnage, des ennemis, ou des éléments visuels comme l'ouverture des coffres. Il facilite la synchronisation des sprites avec les actions du jeu.
- **combat** : Contient la logique de combat entre le joueur et les ennemis, incluant les attaques, les dégâts, les effets visuels et les statistiques associées.

- **entite** : Définit les entités du jeu, telles que le joueur, les ennemis et les PNJ. Ce paquetage comprend les attributs communs (position, santé, mouvements) et permet une gestion polymorphe des comportements.

Ce découpage en paquetages assure une architecture modulaire, dans laquelle chaque sous-système peut être développé, testé et modifié indépendamment. Cela respecte également les principes de conception orientée objet tels que la séparation des responsabilités (SRP) et l'encapsulation.

4 Diagramme de classe

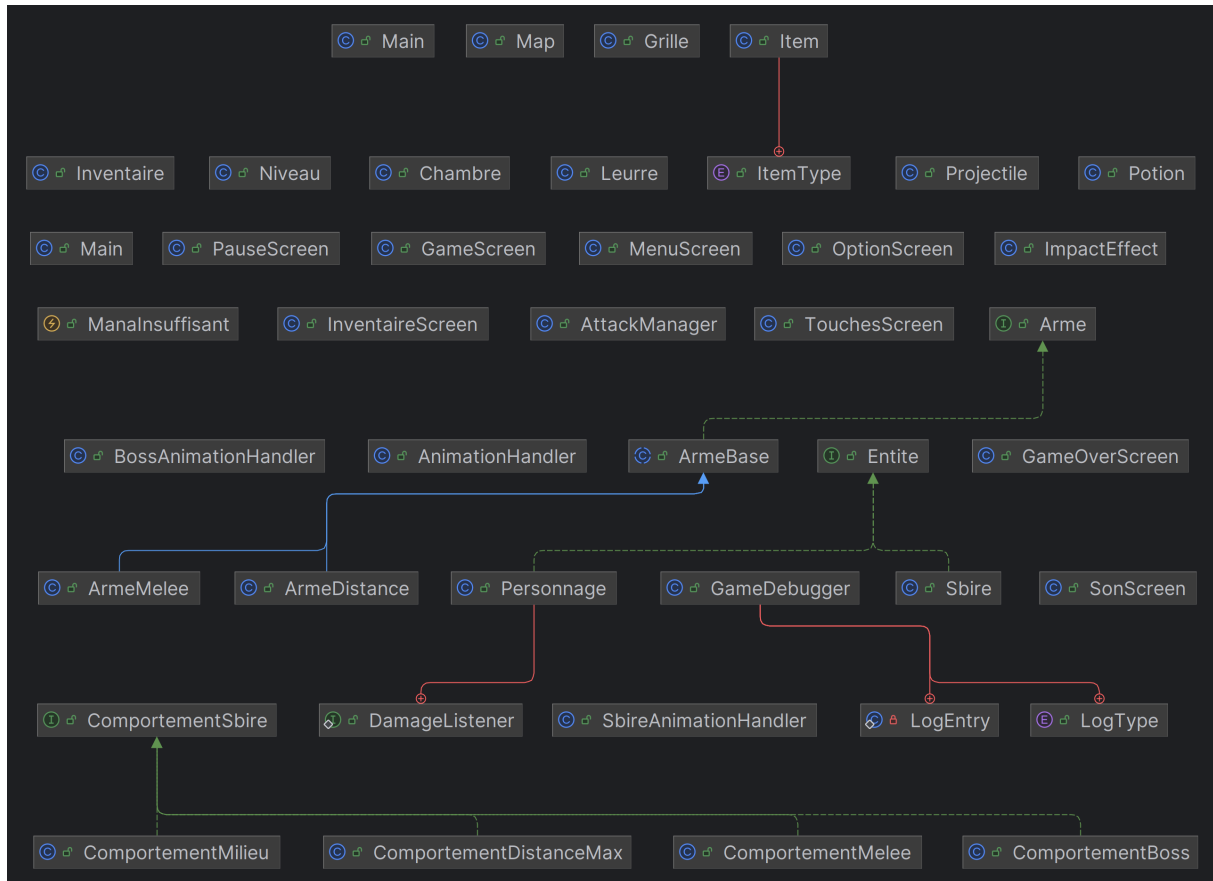


FIGURE 1 – Diagramme UML de l'application sans les méthodes

5 Dynamique du système

Voici comment le système se comporte lors de différents scenarios typiques du jeu :

5.1 Déplacement du joueur sur la carte

- **Déclencheur** : Le joueur appuie sur une des touches de déplacement.
- **Traitement** :
 - Dans la fonction `input()` du `GameScreen`, les touches entrées par le joueur sont captées par le système d'entrée de LibGDX (`Gdx.Input`).
 - La classe `Player` met à jour sa position via sa méthode `changePositionX()` et `changePositionY()`.

- Le système vérifie les collisions à l'aide de `gererCollisionsEntite`, qui vérifie que la hitbox du joueur n'est en contact avec aucune des hitbox des murs de la map.
- La caméra est mise à jour pour suivre le joueur.
- **Résultat** : Le joueur change de position à l'écran en respectant les contraintes de collision.

5.2 Ouverture d'une porte

- **Déclencheur** : Le joueur s'approche d'une porte.
- **Traitement** :
 - Lors du déplacement du joueur, une collision est détectée avec un objet `Porte` durant l'exécution de `gererCollisionsEntite`.
 - Si la porte se trouve à l'indice `i,j` de la map (grille) `map[i,j]` prends la valeur 20 ou 30 (selon l'orientation) pour passer de l'état porte à l'état porte ouverte (plus de collisions).
 - Lors de l'affichage de la map, la texture affichée change pour montrer la porte ouverte
- **Résultat** : Le passage vers une nouvelle salle est débloqué.

5.3 Utilisation d'un objet de l'inventaire

- **Déclencheur** : Le joueur ouvre son inventaire et utilise une potion.
- **Traitement** :
 - Passage à l'écran `InventaireScreen`.
 - Appel de `utiliser()` sur une `Potion`.
 - Mise à jour des attributs du joueur (vie, mana).
- **Résultat** : La potion est consommée et ses effets appliqués.

5.4 Combat contre un sbire

- **Déclencheur** : Le joueur appuie sur la touche Attaque a portée d'un sbire.
- **Traitement** :
 - Utilisation de `AttackManager` pour générer un projectile ou une attaque.
 - Collision avec un `Sbire` détectée.
 - Réduction des PV du Sbire via `subirDegats()`, animation de réaction.
- **Résultat** : Le sbire perd de la vie, potentiellement meurt.

5.5 Mort du joueur

- **Déclencheur** : Les points de vie du joueur tombent à zéro.
- **Traitement** :
 - Appel de `mourir()` dans `Player`.
 - Transition vers un écran de défaite via `Menu` ou `GameScreen`.
- **Résultat** : Le joueur perd la partie, retour au menu principal.

6 Choix de conception, problèmes rencontrés et solutions

6.1 Choix de conception

Dans le cadre du développement de notre jeu, plusieurs décisions ont été prises pour garantir une architecture claire, extensible et modulaire :

- **Architecture modulaire par paquets** : Le code a été découpé en plusieurs paquets logiques (ex : `map`, `inventaire`, `combat`, etc.), chacun correspondant à une responsabilité bien définie du jeu. Ce découpage facilite la maintenance et l'évolution du projet.

- **Utilisation de la programmation orientée objet (POO)** : Chaque entité du jeu (joueur, ennemi, coffre, etc.) est représentée par une classe, ce qui permet l'héritage et la réutilisation de code. L'encapsulation assure une meilleure maîtrise de l'état interne des objets.
- **Génération procédurale des niveaux** : Pour éviter la redondance et rendre chaque partie unique, les cartes sont générées automatiquement via des algorithmes de marche aléatoire et de couplage de chambres.
- **Gestion événementielle et boucle de jeu** : Les interactions utilisateur sont captées via la boucle de rendu de LibGDX, permettant une réactivité en temps réel et une séparation claire entre logique et affichage.

6.2 Problèmes rencontrés et solutions apportés

- **Coffres non visibles sur la carte**
Les coffres étaient générés dans les chambres, mais n'apparaissaient pas sur la carte globale.
Solution : modification de la méthode `placerCoffresDansChambres()` pour recopier explicitement les coffres (valeurs 500) depuis les grilles locales vers la matrice `coord`.
- **Portes qui ne s'ouvrent pas au contact**
Le joueur ne pouvait pas ouvrir certaines portes en les touchant, à cause d'un mauvais calcul des coordonnées.
Solution : correction de l'inversion Y avec `tileY = map.length - 1 - (posY / TILE_SIZE)` et remplacement dynamique de la porte par du sol.
- **Projectiles ennemis mal dirigés**
Les projectiles ne suivaient pas correctement la position du joueur.
Solution : calcul d'un vecteur directionnel normalisé entre l'ennemi et le joueur au moment du tir (`vx = vitesse * dx / norme`).
- **Hitbox mal positionnées ou persistantes**
Certaines hitbox (portes, coffres, projectiles) étaient mal alignées ou restaient actives après interaction.
Solution :
 - Alignement avec `Rectangle(x * TILE_SIZE, y * TILE_SIZE, TILE_SIZE, TILE_SIZE)` ;
 - Suppression dynamique des hitbox obsolètes ;
 - Mise à jour dans les setters de position.
- **Coffres visuellement identiques une fois ouverts**
Aucun changement graphique n'indiquait si un coffre avait été ouvert.
Solution : ajout d'un booléen `estOuvert` et gestion d'une texture différente selon l'état.
- **Texture sans chemin d'origine identifiable**
Il était impossible de retrouver le nom du fichier d'une `Texture` (ex : "gold_chest.png").
Solution : ajout d'un champ `textureChemin` dans la classe `Coffre`, renseigné lors de l'instanciation.
- **Prise en main difficile de LibGDX**
Le framework LibGDX nécessitait une compréhension approfondie difficile sans expérience.
Solution : lecture de la documentation, étude d'exemples existants, et expérimentation progressive sur de petits modules.
- **Manque de vision architecturale au départ**
Difficile de structurer le code proprement sans expérience en développement de jeu.
Solution : séparation en paquetages logiques (`map`, `combat`, `inventaire`, etc.) et application des principes de POO.
- **Problèmes de centrage des hitbox et textures**
Les projectiles et certaines entités n'étaient pas alignés avec leur représentation graphique.
Solution : ajout d'offsets (`position - width/2`) pour centrer à la fois hitbox et texture.

- **Phases du boss trop rapides ou incorrectes**
Les transitions de phases étaient soit trop courtes, soit défailantes.
Solution : ajustement des durées et ajout de logs de debug pour valider le comportement.
- **Leurres mal positionnés et durée trop courte**
Les clones apparaissaient autour du boss au lieu du joueur, et disparaissaient trop vite.
Solution : refonte du système de spawn autour du joueur et augmentation de la durée de vie.
- **Téléportation du boss incohérente**
Distance incorrecte ou apparition dans des zones inappropriées.
Solution : ajustement de la distance et vérification préalable de la validité de la position cible.
- **Manque de séparation pour les entités secondaires (sbires)**
La gestion des sbires du boss encombrait d'autres classes.
Solution : introduction d'une classe Niveau dédiée à leur gestion.
- **Difficulté à travailler en équipe nombreuse avec Git**
La gestion des branches, merges et interfaces a généré des conflits.
Solution : conventions strictes de nommage, travail par fonctionnalité, documentation, et nettoyage avec `git reset --hard origin/branche` si nécessaire.
- **Interfaces de classes mal définies à l'avance**
Cela a causé des dépendances cassées entre les classes des différents membres.
Solution : synchronisation régulière, réunions de clarification, et adaptation rapide des interfaces.
- **Connexion des salles sans surcharger la carte**
La connexion de toutes les chambres générait une surabondance de couloirs.
Solution : algorithme de graphe avec pondération probabiliste (plus une salle a de connexions, moins elle peut en créer).
- **Couloirs trop longs ou déséquilibrés**
Certains chemins étaient visuellement ou mécaniquement inadaptés.
Solution : nettoyage post-génération (recentrage de la carte, suppression des bordures, rotation pour améliorer la disposition).
- **Chambres peu esthétiques ou peu jouables**
Leur structure ne favorisait pas une bonne expérience de jeu.
Solution : refonte du générateur pour équilibrer les zones libres et les murs.
- **Blocage du joueur dans les murs**
Le système de collision pouvait coincer le joueur.
Solution : rollback par axe (annulation du mouvement uniquement sur l'axe de la collision).
- **Spawn d'ennemis en dehors des zones autorisées**
Les mobs apparaissaient parfois en dehors des limites de la carte.
Solution : ajout de vérifications pour empêcher le spawn en zone invalide.
- **Perte de cohérence lors des modifications de la carte**
Le repositionnement de la carte faisait perdre les informations de spawn.
Solution : mise à jour continue des données de spawn pendant les transformations.

Ces solutions nous ont permis d'améliorer la stabilité, la lisibilité du code et la jouabilité du jeu.

7 Organisation de l'équipe et méthodes agiles

7.1 Organisation

7.1.1 Contribution de Mathis

Mathis a apporté des contributions significatives à plusieurs aspects clés du développement de notre jeu vidéo. Ses travaux peuvent être regroupés en plusieurs grands axes :

- **Architecture et Menus du Jeu**
 - Compréhension et mise en œuvre de l'architecture globale du projet.
 - Implémentation des menus, incluant l'écran de pause, la gestion des touches, et la fonctionnalité de non remise à zéro lors de la mise en pause.
- **Gestion des Ressources et Intégration de la Carte**
 - Vérification de la libération des ressources lors de la fermeture du jeu.
 - Intégration de la carte du jeu, facilitant la navigation et l'orientation des joueurs.
- **Améliorations Visuelles et Sonores**
 - Implantation de la mini-carte pour une meilleure orientation dans le jeu.
 - Correction des problèmes liés à la caméra du joueur, améliorant l'expérience visuelle.
 - Intégration des sons in-game pour une immersion audio.
 - Implantation des animations, rendant le jeu plus dynamique.
- **Mécaniques de Jeu**
 - Intégration de la gestion du curseur pour les attaques, améliorant les mécaniques de combat.

Grâce à ces contributions variées, Mathis a joué un rôle essentiel dans l'amélioration de l'expérience utilisateur et a permis au projet d'avancer de manière significative.

7.1.2 Contribution de Raphaël

Raphaël a joué un rôle crucial dans le développement des mécaniques de combat et des effets visuels de notre jeu vidéo. Ses contributions peuvent être regroupées en plusieurs grands axes :

- **Création et Gestion des Classes Armes**
 - Création des classes armes et modification de ces classes.
 - Debug et amélioration des fonctionnalités des armes.
- **Gestion des Mécaniques de Combat**
 - Approche via un `AttackManager` pour gérer les mécaniques de combat en mêlée.
 - Liaison entre les sbires, les niveaux, et les armes.
 - Création d'une classe `Niveau` pour gérer la liste des sbires présents et permettre les collisions entre les sbires et les attaques en mêlée.
- **Gestion des Animations**
 - Gestion des animations des sprites des sbires et du personnage.
 - Création de classes `AnimationHandler` pour animer les déplacements et les attaques des personnages et des sbires.
 - Création de trois handlers différents : un pour le personnage, un pour le boss, et un pour les sbires.
- **Effets Visuels**
 - Ajout d'effets visuels pour rendre le combat plus fluide.
 - Mise en place de signaux visuels pour indiquer quand un ennemi ou le personnage est touché.

La ligne directrice du travail de Raphaël a été axée sur les effets visuels des personnages et des sbires, ainsi que sur les mécaniques de combat côté personnage.

7.1.3 Contribution de Noé

Noé a apporté des contributions significatives à plusieurs aspects du développement de notre jeu vidéo. Ses travaux peuvent être regroupés en plusieurs grands axes :

- **Création et Développement des Sbires**
 - Création de la classe `Sbire` et de ses méthodes.
 - Développement de l'interface `ComportementSbire` et de ses réalisations.
 - Rédaction de la classe `Projectiles`, utile pour les attaques à distance dans la classe `Sbire`.
- **Intégration des Textures et Animations**
 - Intégration des textures sur les sbires.
 - Remplacement des textures par des animations pour améliorer l'aspect visuel.
- **Création des Boss**
 - Création et développement des boss, incluant la modélisation du comportement avec `ComportementBoss`.
 - Implémentation de différentes phases prédéfinies pour les boss : attaque à distance, charge, téléportation, cooldown, chasing.
 - Création de la classe `Leurre` manipulant des clones visuels du boss.
- **Debug et Corrections**
 - Correction des hitbox non centrées sur les textures.
 - Correction des collisions entre projectiles et personnage lors de l'intégration à `GameScreen`.

7.1.4 Contribution de Lilian

Lilian a apporté des contributions significatives à plusieurs aspects du développement de notre jeu vidéo. Ses travaux peuvent être regroupés en plusieurs grands axes :

- **Implémentation des Écrans de Jeu**
 - Implémentation du `GameScreen` pour afficher le personnage dans la carte.
 - Implémentation du `GameOverScreen`.
- **Gestion de l'Inventaire**
 - Implémentation de l'inventaire (classe `Inventory`) avec la gestion de l'`InventoryScreen`.
 - Création de la classe `Item` pour la gestion des objets.
 - Création des items : potions et armes, stockés dans la classe `DatabaseItem`.
- **Création et Gestion des Coffres**
 - Création des coffres et affichage sur la carte.
 - Ouverture des coffres lors du contact avec le joueur, contenant un item aléatoire ajouté à l'inventaire.
- **Création et Gestion du Personnage**
 - Création de la classe `Personnage` qui gère toutes les interactions avec le personnage.
 - Gestion des inputs et des méthodes associées au personnage.

7.1.5 Contribution d'Alexandre

Alexandre a apporté des contributions significatives au développement de notre jeu vidéo, notamment en mettant en place un système de génération procédurale de donjons :

- **Génération Procédurale de Donjons**
 - Création et lissage des salles.
 - Placement des salles et génération d'un réseau de couloirs.
 - Connexion entre les salles avec placement automatique de portes.
 - Optimisation du plan pour raccourcir les chemins et supprimer les couloirs inutiles.
- **Gestion des Collisions et Mécanismes de Porte**
 - Gestion des collisions du joueur avec l'environnement.
 - Implémentation des mécanismes d'ouverture et de fermeture des portes.

- **Gestion des Textures**
 - Gestion des textures pour les murs, sols, portes et objets.
- **Système d’Apparition d’une Horde d’Ennemis**
 - Mise en place du système d’apparition d’une horde d’ennemis.

7.1.6 Contribution de Julian

Julian a apporté des contributions significatives au développement de notre jeu vidéo, notamment en gérant les collisions et en corrigeant des bugs. Ses travaux peuvent être regroupés en plusieurs grands axes :

- **Gestion des Collisions**
 - Création d’une fonction capable de gérer toutes les collisions, implémentée dans le `GameScreen`.
- **Correction de Bugs et Ajustements**
 - Correction de bugs liés aux classes d’armes des personnages.
 - Ajustement des comportements de la classe `Sbire`.
- **Fusion des Branches**
 - Fusion de plusieurs branches pour regrouper les fonctionnalités développées par l’équipe.
- **Intégration des Sbires dans le Jeu**
 - Début d’intégration des sbires dans le jeu (`GameScreen`) : affichage et déplacement.

7.2 Méthodes agiles

Dans le cadre de notre projet de développement de jeu vidéo, nous avons intégré des pratiques agiles pour améliorer notre efficacité et notre réactivité. Bien que nous n’ayons pas suivi un processus agile strict, plusieurs pratiques ont été adoptées pour faciliter la collaboration et la gestion du projet.

7.2.1 Réunions Hebdomadaires

Nous avons instauré des réunions hebdomadaires pour discuter des progrès, des obstacles et des tâches à venir. Ces réunions nous ont permis de maintenir une communication ouverte et de rester alignés sur nos objectifs.

7.2.2 Planification Itérative (Sprints)

Le projet a été divisé en sprints de deux semaines, chacun avec un objectif clair. Cela nous a permis de nous concentrer sur des livrables spécifiques et de mesurer régulièrement nos progrès.

7.2.3 Rétrospectives de Sprint

À la fin de chaque sprint, nous avons organisé des rétrospectives pour réfléchir sur ce qui avait bien fonctionné, ce qui n’avait pas fonctionné, et comment nous pourrions nous améliorer.

7.2.4 Gestion du Backlog

Un tableau de bord a été utilisé pour gérer notre backlog de produit, où toutes les tâches et fonctionnalités à implémenter étaient listées et priorisées.

7.2.5 Collaboration et Communication

Nous avons encouragé une communication ouverte et continue entre tous les membres de l’équipe, utilisant des outils de collaboration en ligne pour partager des mises à jour et des idées.

7.3 Défis et Apprentissages

7.3.1 Défis Rencontrés

- **Adaptation aux Changements** : S'adapter rapidement aux changements de priorités et aux nouvelles idées a été un défi constant.
- **Gestion du Temps** : Respecter les échéances des sprints a parfois été difficile.
- **Communication** : Assurer une communication efficace entre tous les membres de l'équipe a été un défi.

7.3.2 Apprentissages

- **Flexibilité** : Nous avons appris l'importance de rester flexible et ouvert aux changements.
- **Collaboration** : La collaboration et la communication ouverte sont essentielles pour le succès d'un projet agile.
- **Amélioration Continue** : Les rétrospectives nous ont permis de constamment chercher des moyens de nous améliorer.

8 Conclusion

L'intégration de certaines pratiques agiles a grandement bénéficié à notre projet. Ces pratiques nous ont permis de mieux gérer notre temps, d'améliorer notre communication et de rester flexibles face aux changements. Nous continuerons à appliquer et à améliorer ces pratiques dans nos futurs projets.